



**S. B. JAIN INSTITUTE OF
TECHNOLOGY, MANAGEMENT &
RESEARCH, NAGPUR**

Practical No. 07

Aim: Write a Prolog Program to demonstrate the concept of Family relationship.

Name of Student:

Roll No.:

Semester/Year: VI/III

Academic Session:

Date of Performance:

Date of Submission:

AIM: Write a Prolog Program to demonstrate the concept of Family relationship.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- Represent knowledge using **facts**
- Define rules using **logical inference**
- Understand **unification**
- Observe **backtracking**
- Model **hierarchical relationships** (parent, sibling, grandparent, etc.)

THEORY:

Prolog (Programming in Logic) is a **declarative logic programming language** based on:

- First-Order Predicate Logic
- Resolution
- Unification
- Backtracking

Unlike procedural languages (C, Python), Prolog focuses on **what is true**, not **how to compute it**.

Common implementation: **SWI-Prolog**

In Prolog a program consists of:

1. Facts
2. Rules
3. Queries

Installing Prolog (Using SWI-Prolog)

Step 1: Download

Visit: <https://www.swi-prolog.org>

Step 2: Install

Follow OS-specific installer.

Step 3: Run

Open terminal / command prompt:

swipl

?- This is the Prolog query prompt.

Prolog data types:

- **Atoms** → john, apple
- **Variables** → X, Person
- **Numbers** → 10, 3.14
- **Structures** → parent(john, mary)
- **Lists** → [1, 2, 3]

Unification:

Prolog matches patterns for unification. Exa:

parent(john, X) Unifies with: parent(john, mary)

Result:

X = mary

Backtracking:

Prolog automatically searches for alternatives using backtracking.

Example Query:

?- parent(john, X).

Press ; to get next solution.

Important Operators:

Operator	Meaning
:-	If
,	AND
;	OR
=	Unification
is	Arithmetic evaluation
\=	Not unifiable
\+	Negation as failure

A **family relationship system** is a classical example used in Artificial Intelligence to demonstrate:

- Knowledge representation
- Predicate logic
- Recursive reasoning
- Depth-first search with backtracking

ALGORITHM:

1. Start
2. Define family members using male/1 and female/1 facts
3. Define parent relationships

4. Define derived relationships using rules:

- a. father
- b. mother
- c. sibling
- d. grandparent
- e. ancestor

5. Load the program in Prolog

6. Execute queries

7. Observe results and backtracking

8. Stop

PROGRAM CODE:

Create a file named: family.pl

% ----- Facts -----

male(john).

male(sam).

male(paul).

female(mary).

female(anna).

female(lisa).

parent(john, sam).

parent(john, anna).

parent(mary, sam).

parent(mary, anna).

parent(sam, paul).

parent(lisa, paul).

% ----- Rules -----

father(X, Y) :-
 parent(X, Y),
 male(X).

mother(X, Y) :-
 parent(X, Y),
 female(X).

sibling(X, Y) :-
 parent(Z, X),
 parent(Z, Y),
 X \= Y.

grandparent(X, Y) :-
 parent(X, Z),
 parent(Z, Y).

ancestor(X, Y) :-
 parent(X, Y).

ancestor(X, Y) :-
 parent(X, Z),
 ancestor(Z, Y).

INPUT & OUTPUT:


```

SWI-Prolog (AMD64, Multi-threaded, version 9.2.6)

File Edit Settings Run Debug Help

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- consult('c:/Users/KARAN PRAJAPATI/OneDrive/Desktop/family.pl').
true.

?- father(X, sam).
X = john.

?- sibling(sam, X).
X = anna.

?- grandparent(X, paul).
X = john ;
X = mary.

?- ancestor(X, paul).
X = sam ;
X = lisa ;
X = john ;
X = mary.

~ -

```

Step 1: Start Prolog

swipl

Step 2: Consult File

?- consult('family.pl').

Output:

True**Queries & Output****1. Find father of Sam**

?- father(X, sam).

Output:

X = john.

2. Find siblings of Sam

?- sibling(sam, X).

Output:

X = anna.

3. Find grandparents of Paul

?- grandparent(X, paul).

Output:

X = john ;

X = mary.

4. Find ancestors of Paul

?- ancestor(X, paul).

Output:

X = sam ;

X = lisa ;

X = john ;

X = mary.

(Uses recursion + backtracking)

CONCLUSION:

DISCUSSION QUESTIONS:

1. What is a fact in Prolog?
2. What is the difference between fact and rule?
3. What is unification?
4. Explain backtracking with example.
5. Why is recursion used in `ancestor` rule?
6. What is the difference between `=` and `is`?

REFERENCES:

- <https://www.swi-prolog.org/>
- https://www.tutorialspoint.com/prolog/prolog_introduction.htm
- <https://www.geeksforgeeks.org/artificial-intelligence/prolog-an-introduction/>
- https://www.youtube.com/watch?v=dKn-BbS_zOO